

Matricola: _____

Nome: _____

Cognome: _____

ESAME Programmazione Avanzata e Parallela

3 febbraio 2025

L'esame consiste di 10 domande a risposta multipla sugli argomenti del corso. Ogni domanda può ricevere un punteggio massimo di *tre* punti. Affinché una risposta sia considerata valida la scelta *deve essere motivata*. Una risposta errata o non motivata riceverà *zero* punti.

Domanda 1

Si supponga di avere il seguente file makefile:

```
CC = gcc
```

```
CFLAGS = -Wall -O3
```

```
SRCS = A.c B.c C.c D.c
```

```
OBJS = A.o B.o C.o D.o
```

```
PROGRAM = program
```

```
all: $(PROGRAM)
```

```
$(PROGRAM): $(OBJS)
```

```
$(CC) $(CFLAGS) -o $@ $^
```

```
%.o: %.c
```

```
$(CC) $(CFLAGS) -c $< -o $@
```

```
clean:
```

```
rm -f $(OBJS) $(PROGRAM)
```

```
.PHONY: all clean
```

Se, dopo aver precedentemente invocato `make` viene modificato il file `A.c` e rimosso il file `D.o`, alla successiva invocazione di `make` quali comandi saranno eseguiti

<code>gcc -Wall -O3 -c A.c -o A.o</code>	<code>gcc -Wall -O3 -o program A.o B.o C.o D.o</code>
☐ <code>gcc -Wall -O3 -c D.c -o D.o</code>	☐ <code>gcc -Wall -O3 -c D.c -o D.o</code>
☐ <code>gcc -Wall -O3 -o program A.o B.o C.o D.o</code>	☐ <code>gcc -Wall -O3 -o program A.o B.o C.o D.o</code>

Domanda 2

Si supponga di avere questo frammento di codice C:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (M[i * n + j] == 1) {
            ones++;
        } else {
            zeros++;
        }
    }
}
```

e si assuma di avere variabili `M`, `zeros`, `ones` definite e del tipo corretto. La variabile `M` punta a un vettore di n^2 elementi che possono essere solo zero o uno scelti casualmente con uguale probabilità.

Quale delle seguenti affermazioni è corretta? Se necessario, si assuma che il compilatore non ottimizzi.

- | | |
|--|--|
| <p>Una versione branchless del corpo del ciclo più interno è:</p> <p>☐ <code>int tmp = M[i * n + j] == 1;</code>
 <code>ones += tmp;</code>
 <code>zeros += (1-tmp);</code></p> <p>Il branch predictor è in grado di riconoscere questa tipologia di pattern su matrici, quindi ci aspettiamo una accuratezza molto elevata</p> | <p>Una versione branchless del corpo del ciclo più interno è:</p> <p>☐ <code>ones += M[i * n + j] == 1;</code>
 <code>zeros += -1 * ones;</code></p> <p>Invertendo l'ordine dei cicli avremmo migliore località di memoria per l'accesso a <code>M</code></p> |
|--|--|
-
-
-

Domanda 3

Dato il seguente codice:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        v[i] += M[j * n + i];  
    }  
}
```

Supponendo che le variabili v e M siano correttamente definite, con M una matrice salvata in *row-major order*, quale delle seguenti affermazioni è corretta?

- | | |
|--|--|
| ☐ La località di memoria sarebbe peggiore se M fosse salvata in <i>column-major order</i> | ☐ I branch sarebbero meglio predicibili se M fosse salvata in <i>column-major order</i> |
| ☐ La località di memoria sarebbe migliore se M fosse salvata in <i>column-major order</i> | ☐ La località di memoria sarebbe migliore se i due cicli <code>for</code> fossero collassati in uno solo con n^2 iterazioni |
-
-
-

Domanda 4

Indicare quale delle seguenti non è una ottimizzazione del compilatore:

- | | |
|--|--|
| ☐ loop unrolling | ☐ function inlining |
| ☐ structural hazard | ☐ propagazione delle costanti |
-
-
-

Domanda 5

Quale delle seguenti affermazioni su `mmap` è corretta?

- | | |
|--|---|
| ☐ Se non scegliamo dove effettuare il mapping, questo viene fatto sempre all'indirizzo di memoria 0 | ☐ Quando viene effettuato <code>mmap</code> di un file il contenuto del file viene letto per intero e caricato immediatamente in memoria |
| ☐ Per accedere al contenuto del file è sufficiente il puntatore ritornato da <code>mmap</code> | ☐ Il contenuto visibile in memoria è solo quello già letto in precedenza con <code>read</code> , le rimanenti locazioni se lette restituiscono 0 |
-
-
-

Domanda 6

Dato il seguente codice C facente uso di OpenMP:

```
int main(int argc, int * argv[])
{
    const int n = 10000;
    int * M1 = random_matrix(n, n);
    int * M2 = random_matrix(n, n);
    int v = 0;
    /* TODO */
    for (int i = 0; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (M2[i*n + j] > M1[i*n + j]) {
                swap(&M2[i*n + j], &M1[i*n + j]);
#pragma omp critical
                v += 1;
            }
        }
    }
    printf("%d\n", v);
    return 0;
}
```

Si supponga che le funzioni `random_matrix` e `swap` siano correttamente definite. Al posto di `TODO` che codice **non permetterebbe di ottenere** il risultato atteso (i.e., `v` contiene il numero di valori di `M2` che sono maggiori dei valori di `M1` in uguale posizione)?

☐ #pragma omp parallel

☐ #pragma omp parallel for

☐ #pragma omp parallel for collapse(2) ☐ nulla

Domanda 7

Dato il seguente codice Python:

```
class A:
```

```
    def f(self, x):  
        return x + 1
```

```
class B(A):
```

```
    def f(self, x):  
        return 3*x
```

```
class C(A):
```

```
    def f(self, x):  
        return super().f(x+2)
```

```
class D(B):
```

```
    pass
```

```
x = D()
```

```
y = C()
```

```
print(x.f(4) + y.f(10))
```

cosa viene stampato a schermo?

☐ 25

☐ 37

☐ 42

☐ 20

Domanda 8

Dato il seguente codice Python:

```
def f(g):  
    def h(k):  
        return lambda x: k(g(x))  
    return h
```

```
func = f(lambda x: x+10)(lambda y: 2*y)
```

```
print(func(3))
```

Cosa viene stampato a schermo?

☐ 13

☐ 26

☐ 16

☐ `<function f.<locals>.h.<locals>.<lambda>
at 0x103008ea0>` (l'indirizzo può essere
differente)

Domanda 9

Dato il seguente codice Python:

```
def d1(h):  
    def f(*args, **kwargs):  
        return h(*args, **kwargs) + 3  
    return f
```

```
def d2(h):  
    @d1  
    def f(*args, **kwargs):  
        return 2*h(*args, **kwargs)  
    return f
```

```
@d2  
def f(x, y):  
    return x+2*y
```

```
print(f(3, 4))
```

Quale è il valore stampato a schermo?

☐ 11

☐ 28

☐ 22

☐ 25

Domanda 10

Dato il seguente codice Python:

```
def f(m):  
    for n in range(2, m+1):  
        p = True  
        for i in range(2, n):  
            if n % i == 0:  
                p = False  
        if p:  
            yield n
```

```
pr = f(100)  
for i in pr:  
    print(i)
```

Quali sono i valori stampati a schermo?

☐ <generator object f at 0x102b39c60>
☐ ripetuto 100 volte

☐ 2 3 4 5 6...99 100

☐ 2 3 5 7 11...89 97

☐ 2 5 10 20 50 100
